

CA OS For Exam Notes

Computer Architecture and Operating Systems — For Exam

Block 1: Basic Functional Architecture

1.1.1 Generations of Computers ★★

For Exam

Include all five generations with **period, technology, one feature, and one example**.

1. **First generation (1940–1956):** Vacuum tubes and magnetic drums. Large, costly, high heat.
Examples: UNIVAC, ENIAC, EDVAC.
2. **Second generation (1956–1963):** Transistors. Smaller, faster, less heat. High-level languages COBOL and FORTRAN. Examples: IBM 1400 series, IBM 7090.
3. **Third generation (1964–1971):** Integrated Circuits (ICs). More reliable, faster, smaller, cheaper.
Examples: IBM System/360, PDP-11.
4. **Fourth generation (1971–2010):** VLSI / microprocessors. Personal computers and laptops.
Examples: Apple, CRAY-1.
5. **Fifth generation (present and future):** Artificial Intelligence. Systems that try to think and act like humans. Example: PARAM 10000.

Also write: von Neumann's five units (input, output, memory, ALU, control) remain the same in all generations.

1.1.2 Functional Units of a Computer ★★★

For Exam

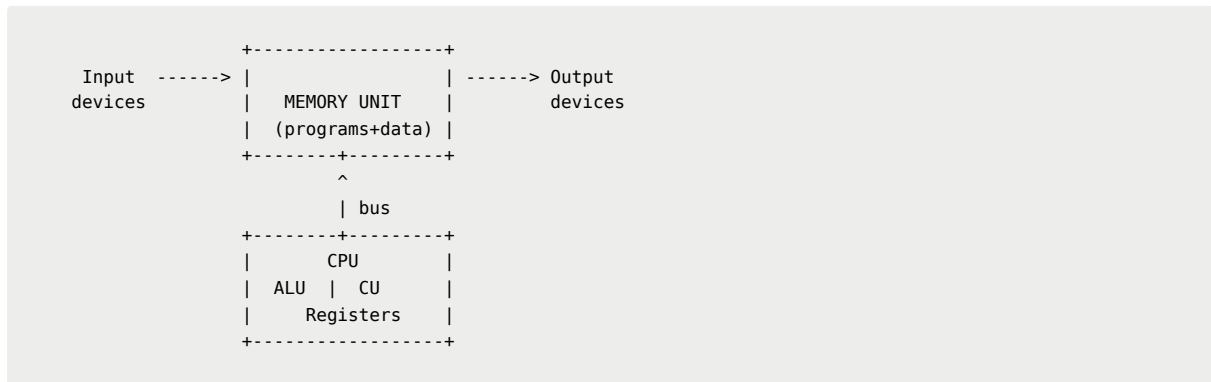
Include: von Neumann introduction, all **five units** with role of each, how they work together, CPU definition, and the block diagram.

Introduction. According to John von Neumann's stored-program model, a computer has five functional units linked by buses: Input, Output, Memory, ALU, and Control unit. **CPU = Control unit + ALU + Registers.**

1. **Input unit:** Accepts data and programs (keyboard, mouse, scanner, disk) and converts them into binary form.
2. **Memory unit:** Stores instructions, data, and results. Primary memory (RAM/ROM) is fast; secondary memory (disk) is permanent and slower. Programs must reside in main memory during execution.
3. **ALU:** Performs arithmetic (+, −, ×, ÷) and logical/comparison operations. Work area of the CPU.
4. **Control unit:** Nerve centre. Fetches and interprets instructions and generates control signals and timing for all other units.
5. **Output unit:** Presents results (monitor, printer, speaker).

Together: Input → Memory → Control unit sequences fetch–decode–execute → ALU processes → results to memory and output.

Conclusion. Each unit has a distinct role; only their coordinated operation makes a complete computer.



1.1.2.2 Primary and Secondary Memory ★★

For Exam

Memory stores code, data and results. Write **primary memory** first, then **secondary memory**.

Primary memory

1. Fast semiconductor storage (RAM and ROM).
2. RAM is volatile; ROM is non-volatile firmware.
3. Expensive compared with disk.
4. Programs must reside here while they run.
5. Examples: RAM, ROM.

Secondary memory

1. Slow, cheaper, permanent storage.
2. Non-volatile.
3. Used for bulk storage; supplements primary memory.
4. Examples: hard disk, optical disk.

1.1.4 Bus Types ★★

For Exam

Include: definition of bus, internal vs external, and the **three system-bus lines**.

A bus is a shared set of lines connecting CPU, memory and peripherals.

1. **Internal (system) bus** — connects processor, memory and I/O.
2. **External bus** — connects peripherals (keyboard, mouse) to the CPU.

System bus has three types of lines:

1. **Data bus** — bi-directional; carries data.

2. **Address bus** — unidirectional; carries addresses.
3. **Control bus** — read/write and other control signals.

Also mention: lines may be **dedicated** or **multiplexed**. Usual I/O connection is a **single / common bus**.

1.2.1 Registers and Decoder ★★

For Exam

Registers — include: definition + at least 5 uses.

A register is a small, fast CPU storage made of flip-flops.

- **ACC** — intermediate arithmetic results
- **PC** — address of the next instruction
- **IR** — current instruction
- **MAR** — memory address
- **MBR / DR** — data to/from memory
- **SP** — top of stack

Decoder — include: it translates encoded binary inputs into a control line. Uses: decode **opcode** into control signals; **address decoding** to select memory/I/O; convert sequence-counter output into timing signals **T0–T15**.

1.2.2 Hardwired Control Unit ★★

For Exam

Include: one-line meaning + **key components with function** + one comparison line with microprogrammed control.

Hardwired control generates signals using fixed digital circuits. Fast, but hard to modify.

1. **IR** — holds the current instruction.
2. **Opcode decoder** (3×8) — opcode → D0–D7.
3. **I-bit flip-flop** — indirect-addressing bit.
4. **Sequence counter** — timing states under the clock.
5. **Timing decoder** (4×16) — T0–T15.
6. **Control logic gates** — combine decoder, timing and IR bits into final control signals.
7. **Master clock** — synchronises the unit.

Microprogrammed control: control words stored in ROM; slower, easier to change. **CMAR** holds the address of the next microinstruction.

1.3.1 Need for Addressing Modes ★★

For Exam

Include the **definition** and these **six needs**:

Addressing modes are methods of specifying where operands are (instruction, register, or memory).

1. **Pointers** to memory addresses
 2. **Loop control** using counters
 3. **Indexing** of arrays and tables
 4. **Program relocation**
 5. **Shorter address field** in instructions
 6. **Efficient, flexible** assembly programs
-

1.3 Addressing Modes with Examples ★★★

For Exam

Include: definition, **all eight modes with one example each**, and the six needs (short). Draw: instruction = [OPCODE][ADDRESS FIELD].

1. **Implied:** operand hidden in opcode (often ACC). Examples: `CLC`, `NOP`.
2. **Immediate:** data is inside the instruction. Example: `MOV AX, 40H`.
3. **Direct:** address field = memory address of operand. Example: `LDA 2050`.
4. **Register:** operand is in a named CPU register. Example: `ADD R1, R2`.
5. **Register indirect:** register holds the effective address. Example: `MOV A, M`.
6. **Auto-increment / Auto-decrement:** register-indirect plus automatic update. Used for tables.
7. **Indirect:** instruction points to a location that stores the effective address. Example: `ADD @200H`.
Extra memory access.
8. **Indexed:** EA = base + index register. Example: `MOV AX, [SI+05]`. Used for arrays.

Need: pointers, loops, indexing, relocation, shorter address field, efficiency.

Direct vs Indirect Addressing ★★

For Exam

Write **direct addressing** first, then **indirect addressing**.

Direct addressing

1. Address field of the instruction is the address of the operand.
2. EA = address in the instruction.
3. One memory access to get the operand, so it is faster.
4. Example: `LDA 2050`.

Indirect addressing

1. Address field points to a memory location that stores the effective address (a pointer).
2. EA = M[address field].
3. Extra memory access, so it is slower.

4. Example: `ADD @200H`.

Summary: Direct points straight to data. Indirect points to a pointer.

Indexed Addressing Mode ★★

For Exam

Include: formula, how it works, why it is useful, one example.

EA = base / displacement + contents of index register.

The instruction gives a fixed base and names an index register. CPU adds them to get the operand address. Base stays fixed; index changes in a loop to visit array elements.

Significance: arrays, tables, repetitive data access without rewriting the address each time.

Example: `Load R4, 4(R2)`; `MOV AX, [SI+05]`. If base = 2800H and index = 01H, EA = 2801H.

1.4 Program Control Instructions ★★

For Exam

Include the **types**, then a short meaning of unconditional branch, conditional branch, and subroutine.

Types:

1. Unconditional branch / jump
2. Conditional branch
3. Subroutine call and return
4. Halt
5. Interrupt / skip

Unconditional branch: always loads a new address into PC (`JMP`). No flags tested.

Conditional branch: jumps only if a flag is true (zero, carry). Used for if-conditions and loops.

Subroutine: reusable procedure. CALL saves return address; RETURN comes back to the caller.

NOP: does nothing; used for delay or as a placeholder.

Block 2: I/O and DMA

2.1 RTL and Common Bus ★★

For Exam

Include: RTL meaning, one transfer example, fetch-cycle steps, and role of multiplexer.

RTL = Register Transfer Language. It describes micro-operations such as `R2 ← R1` (copy R1 to R2 when the control signal is enabled).

Fetch cycle:

1. `MAR ← PC`
2. Memory read; instruction → IR
3. `PC ← PC + 1`

Multiplexer on a common bus: selects which register is placed on the bus using select control signals, so many registers share one bus.

2.2 Functions of I/O Interface ★★ / ★★★

For Exam

Include these **four functions**, each with one line of explanation. For 15 marks, add an example under each and a short introduction/conclusion.

Peripherals differ from the CPU in speed, format and operation. The I/O interface sits between the processor bus and the device.

1. **Data transfer** — moves data between CPU/memory and the device through ports/registers.
2. **Control signal management** — start, stop and coordinate the peripheral (control / status / data-in / data-out).
3. **Data conversion** — converts signal levels and formats (e.g. parallel CPU data ↔ serial UART).
4. **Synchronisation** — matches fast CPU and slow device using status, handshake and buffers so data is not lost.

Also include if asked:

- **Polling:** CPU repeatedly checks device status; simple but wastes CPU time.
 - **Synchronous I/O:** shared clock.
 - **Asynchronous I/O:** handshake / ready-busy; better when speeds differ.
 - Two interface types: **Memory-mapped I/O** and **Isolated (port-mapped) I/O**.
-

2.3 Types of Interrupts / Priority Interrupt ★★

For Exam

Include: meaning of priority interrupt + the types listed below.

Priority interrupt: when two or more devices request at once, the **highest-priority** request is served first. Fast devices (disk) get high priority; slow devices (keyboard) get low priority. Priority is set by polling (software) or daisy-chain / parallel encoder (hardware).

Types to include:

1. **Hardware interrupts** — from external devices.
 - **Maskable** — can be delayed.
 - **Non-maskable** — must be handled immediately.
1. **Software interrupts** — from program/system.
 - **Normal** — system call / interrupt instruction.
 - **Exception** — unplanned (divide by zero).

```
Device1 (High) ----\
Device2          --> Priority logic --> CPU --> Save PC --> ISR --> Return
Device3 (Low)  ----/
```

2.4 Direct Memory Access (DMA) ★★★

For Exam

Include: concept, need, working steps, **three modes**, **advantages**, and the diagram.

Concept. DMA lets a device transfer a **block** of data **directly between I/O and main memory** without the CPU moving each byte. The **DMA controller** becomes bus master.

Need. Programmed I/O is too slow for disks, network and multimedia.

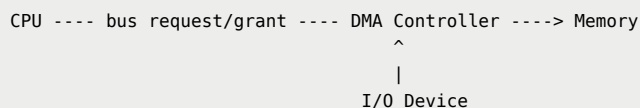
Working.

1. Device / CPU requests DMA.
2. DMAC raises **bus request**; CPU gives **bus grant**.
3. CPU sets starting address, count and direction.
4. DMAC transfers data memory ↔ device.
5. Count reaches zero → release bus → **interrupt CPU**.

Modes.

- **Burst** — hold bus till the whole block is done.
- **Cycle stealing** — one word, then return the bus.
- **Transparent** — use the bus only when CPU is idle.

Advantages. Faster bulk transfer; CPU is free; higher throughput; one interrupt per block instead of per byte.



Block 3: Parallel Computer Structures

3.1 Parallel vs Serial Computing ★★

For Exam

Write **serial computing** first, then **parallel computing**.

Serial computing

1. Uses one processing element (PE).
2. Executes one instruction after another.
3. Slower for large problems.
4. Example: uniprocessor / SISD.

Parallel computing

1. Uses many processing elements.
2. Several operations run at once.
3. Faster for large problems.

4. Example: multiprocessor, array processor.
-

3.2 Flynn's Classification ★★★

For Exam

Include: year and basis (instruction stream × data stream), all **four classes with meaning + example**, and a conclusion. Draw one small structure sketch for each class if space allows.

Introduction. M. J. Flynn (1966) classified computers by multiplicity of instruction streams and data streams: SISD, SIMD, MISD, MIMD.

1. **SISD** — single instruction, single data. Uniprocessor. Example: conventional PC.
2. **SIMD** — single instruction, multiple data. One CU broadcasts to many PEs. Example: array processor, ILLIAC IV, GPU-style SIMD.
3. **MISD** — multiple instruction, single data. Rare commercially.
4. **MIMD** — multiple instruction, multiple data. Example: multicore, multiprocessor, cluster.

Conclusion. SISD is serial; SIMD and MIMD are the practical parallel classes; MISD is uncommon.

UMA vs NUMA ★★

For Exam

Write **UMA** first, then **NUMA**.

UMA (Uniform Memory Access)

1. All processors see shared memory with equal access time.
2. Typical of SMP systems.
3. Simpler to program.
4. Harder to scale because of bus/memory contention.

NUMA (Non-Uniform Memory Access)

1. Local memory is faster than remote memory.
2. Used in scalable multiprocessors.
3. Programming must consider locality.
4. Scales better than UMA.

Summary: UMA is simpler but limited in scale. NUMA scales better.

3.3 Pipelining ★★

For Exam

Include: definition, stages, processor cycle, one advantage, one disadvantage.

Pipelining overlaps execution of successive instructions, like an assembly line. Typical stages: Fetch → Decode → Execute → Memory → Write-back. After the pipeline fills, about one instruction completes per cycle.

Processor cycle (tp): time to advance **one stage** (slowest stage + latch). For k stages and n tasks, time ≈ $[k + (n - 1)] \times t_p$.

Also include if asked:

- **ILP need:** higher performance, better unit utilisation, more throughput, schedule independent instructions; clock speed alone is not enough.
 - **Pipeline bubble:** empty stall cycle due to data/control/structural hazard.
 - Types: arithmetic pipeline, instruction pipeline, processor pipeline.
-

3.4 Performance Measures of Vector Processing ★★

For Exam

Include: two overheads + **four measures**.

Overheads: (i) **Setup time** — routing the vector to the functional unit; (ii) **Flushing time** — until the first result leaves the pipeline.

Measures:

1. **Improve the vector instruction** — fewer memory accesses, better use of units.
2. **Integrate scalar instructions** — group similar scalars so the pipeline is not reconfigured often.
3. **Algorithm** — choose algorithms that expose long vectors.
4. **Vectorising compiler** — recover parallelism from high-level code ($A \rightarrow L \rightarrow O \rightarrow M$).

Also mention FLOPS / megaflops on supercomputers.

Block 4: Operating Systems

4.1 Process Management ★★

For Exam

Include these **six OS duties** and mention PCB.

1. Create / delete processes (allocate PCB, load program, assign PID).
2. Schedule CPU (long-term, short-term, medium-term schedulers and dispatcher).
3. Suspend / resume (ready, waiting, running; swapping if needed).
4. Synchronisation so shared data is not corrupted.
5. IPC — pipes, messages, shared memory, sockets.
6. Termination — reclaim CPU, memory, open files.

PCB stores state, PC, registers, scheduling info and memory info.

OS = resource manager + interface between programs and hardware.

Six components of OS: process, memory, file, mass storage, I/O, protection/security.

4.1 Types of OS / Time-Sharing ★★

For Exam

Include the **four SLM types**, then extra detail on time-sharing.

1. **Serial processing** — programmer talks to hardware; one user after another; no modern OS.

2. **Simple batch** — operator groups similar jobs; a **monitor** runs them in sequence.
3. **Multiprogrammed batch** — when one job waits for I/O, CPU switches to another job (CPU kept busy).
4. **Time-sharing** — CPU given to users in short **time slices / quanta**. Each user feels the CPU is theirs.

Time-sharing — also include: interactive response, many programs in memory, frequent context switches, fair sharing. Issues: reliability, security, integrity, communication.

4.1 Layered Structure of OS ★★

For Exam

Include: layer 0 and layer N, the rule “uses only lower layers”, advantages, and one line each on simple structure and microkernel.

```
Layer N = User interface
...
Layer 1
Layer 0 = Hardware
```

Each layer has defined inputs, outputs and functions and uses **only lower layers**.

Advantages: modularity, easier debugging and modification, clear abstraction; lower layers hide hardware.

Simple structure (MS-DOS): poor module boundaries; a user-program fault can crash the system.

Microkernel (Mach): kernel keeps only address spaces, threads, IPC; other services run in user space and communicate by messages. More reliable and secure.

4.2 OS Services ★★

For Exam

Include the **nine services** by name, with one short line each if space allows.

1. User interface — CLI / GUI / AUI
2. Program execution — load program, create process
3. Resource allocation — CPU, memory, devices
4. I/O operations
5. File-system manipulation
6. Communication (IPC)
7. Error detection
8. Accounting
9. Protection and security

System calls (if 4 marks on process control): `fork()` create child; `exec()` replace image; `wait()` parent waits; `exit()` terminate.

4.3 Process vs Program ★★

For Exam

Write **program** first, then **process**.

Program

1. Passive entity — a set of instructions stored as a file on disk.
2. Contains code only.
3. Remains until the file is deleted.
4. One program file.

Process

1. Active entity — a **program in execution**.
2. Contains code + PC + registers + stack + data + PCB.
3. Created, scheduled, may wait, then terminates.
4. Many processes can run from the same program.

Example: Word.exe on disk is a program. Opening Word creates a process.

4.3 Process State Diagram ★★★

For Exam

Include: introduction, **five states**, **all transitions**, PCB/context switch, labelled diagram, conclusion.

Introduction. In a multiprogramming OS a process changes state. The diagram shows legal transitions.

States.

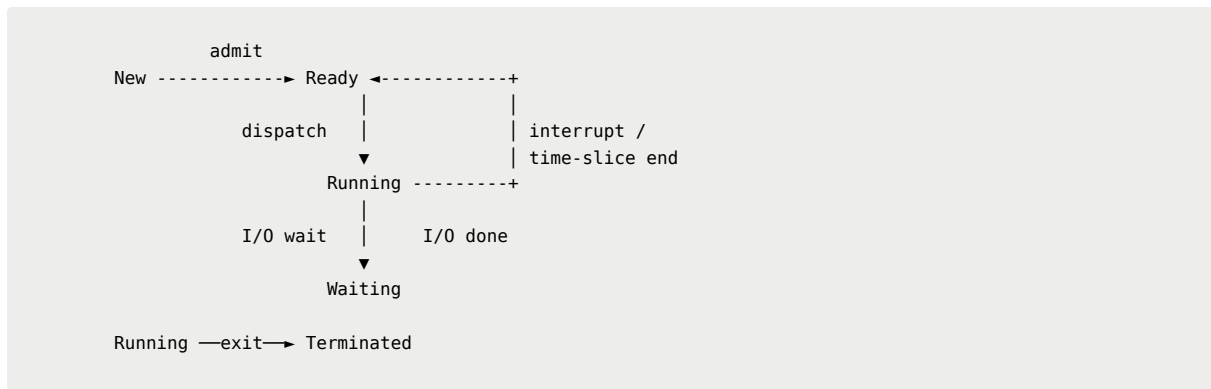
1. **New** — being created; PCB allocated.
2. **Ready** — waiting for CPU; in ready queue.
3. **Running** — executing on CPU.
4. **Waiting (Blocked)** — waiting for I/O or an event.
5. **Terminated** — finished; resources reclaimed.

Transitions.

- New → Ready: **admitted**
- Ready → Running: **short-term scheduler / dispatcher**
- Running → Ready: **interrupt / time-slice end**
- Running → Waiting: **I/O or event wait**
- Waiting → Ready: **I/O or event done**
- Running → Terminated: **exit**

Context switch: save old PCB (PC, registers), load new PCB.

Schedulers (include if asked): long-term = which job enters memory; **short-term** = which ready process gets CPU; **medium-term** = swapping.



4.3 Priority vs Non-priority / Preemptive vs Non-preemptive ★★

For Exam

Write **non-preemptive (non-priority)** first, then **preemptive (priority)**. Same points cover both wordings of the question.

Non-preemptive / non-priority scheduling

1. Running process keeps the CPU until it finishes or blocks for I/O.
2. Switch happens only on terminate or wait.
3. Examples: **FCFS, SJF**.
4. Simple, fewer context switches.
5. Disadvantage: convoy effect; important jobs may wait.

Preemptive / priority scheduling

1. CPU can be taken away from a running process.
2. Switch also happens on higher priority, shorter remaining time, or time quantum.
3. Examples: **SRTF, Round Robin**.
4. Better response for urgent and interactive work.
5. Disadvantage: more context switches; low-priority processes may starve.

Summary: Non-preemptive never forcibly seizes the CPU. Preemptive may seize it.

Formulas for numerical (15 marks):

$$TAT = CT - AT$$

$$WT = Start - AT \text{ (or } TAT - BT)$$

$$Avg\ WT = (\text{sum of } WT) / n$$

Always draw a **Gantt chart**.

FCFS Numerical ★★★

For Exam

Include: definition (order by arrival, non-preemptive), Gantt chart, waiting time of each process, average.

Example: P1(0,4), P2(2,5), P3(3,3), P4(4,2), P5(5,1)

Order: P1 → P2 → P3 → P4 → P5

Gantt: 0–4 P1, 4–9 P2, 9–12 P3, 12–14 P4, 14–15 P5

WT = Start – AT: 0, 2, 6, 8, 9

Avg WT = $25/5 = 5$

SJF Numerical ★★★

For Exam

Include: among ready jobs pick **smallest burst** (non-preemptive), Gantt, WT, average.

Example: P1(1,7), P2(2,5), P3(3,1), P4(4,2), P5(5,8)

P1: 1–8 (only ready at t=1)

Then shortest ready is P3: 8–9

Then P4: 9–11

Then P2: 11–16

Then P5: 16–24

WT: 0, 9, 5, 5, 11

Avg WT = $30/5 = 6$

Block 5: Process Synchronization

5.1 Interprocess Communication (IPC) ★★ / ★★★

For Exam

Include: two models + **all six methods** with one line each. For 15 marks, expand each method and add Dining Philosophers.

Models: Shared memory | Message passing.

1. **Pipes** — unidirectional; related processes (parent–child).
2. **Named pipes (FIFO)** — bidirectional; unrelated processes that know the name.
3. **Message queues** — stored until the receiver reads; sender and receiver need not be active together.
4. **Shared memory** — fastest; needs semaphore/mutex.
5. **Semaphores** — integer wait/signal for synchronisation, not bulk data.
6. **Sockets** — network client–server IPC.

Dining Philosophers (15-mark second part) — include:

Five philosophers, five chopsticks, each needs **two** to eat. If all pick the left chopstick at once, each waits forever for the right one.

Four conditions: mutual exclusion, hold and wait, no preemption, circular wait.

Solutions: pick both only if both free; odd/even different order; at most four philosophers try at once.

Draw five people in a circle.

5.4 Deadlock ★★

For Exam

Include: definition, **four conditions**, and **one example**.

Deadlock: a set of processes is blocked forever; each holds a resource and waits for a resource held by another in the set.

Four conditions (all must hold):

1. Mutual exclusion
2. Hold and wait
3. No preemption
4. Circular wait

Break any one condition → no deadlock.

Example: P1 holds tape T and wants printer Pr; P2 holds Pr and wants T. Circular wait.

Hold and wait (if asked separately): process holds at least one resource while waiting for more.

Prevention: request all at once, or release held resources before requesting new ones.

5.4 Banker's Algorithm ★★

For Exam

Include: it is **deadlock avoidance**; Max / Allocation / Need / Available; **request steps**; meaning of **safe sequence**.

Need = Max – Allocation.

When P_i requests resources:

1. Request $\leq$ Need? else **error**.
2. Request $\leq$ Available? else **wait**.
3. Tentatively allocate (Available –, Allocation +, Need –).
4. Run **safety algorithm**. If a **safe sequence** exists, grant; else undo and wait.

Safe state = there is an order in which every process can finish with currently available resources plus what earlier processes will release. Like a banker who never lends so much that remaining cash cannot cover every customer's maximum claim.

Block 6: Memory Management and File Systems

6.1 Logical vs Physical Address ★★

For Exam

Write **logical address** first, then **physical address**. Also mention MMU.

Logical address

1. Generated by the CPU.
2. Also called virtual address.
3. This is the address seen by the program.

Physical address

1. Actual location in RAM after translation.
2. Also called real address.
3. Formed by MMU: $\text{physical} = \text{logical} + \text{base}$, checked against **limit**.

MMU = Memory Management Unit. Binding may be at compile time, load time, or execution time.

6.1 Contiguous Allocation / Best-Fit ★★

For Exam

Include: meaning of contiguous; **internal + external fragmentation**; placement methods; best-fit in more detail.

Contiguous allocation gives a process **consecutive** locations (fixed or dynamic partitions).

Disadvantages: internal fragmentation (waste inside a partition) and external fragmentation (scattered holes).

Placement: First-fit, Next-fit, Best-fit, Worst-fit.

Best-fit: choose the **smallest hole that is large enough**. Leaves a small leftover. Advantage: does not split a very large hole unnecessarily. Disadvantage: leftover holes may be too tiny to use; search is slower.

Compaction: slide processes together to make one big free hole; reduces external fragmentation; costly.

6.2 Paging vs Segmentation ★★★

For Exam

Write **paging** first, then **segmentation**. For 15 marks, part (b) is page-fault counting.

Paging

1. Process is split into fixed-size **pages**; memory into same-size **frames**.
2. Address = (page number, offset).
3. Page table maps page → frame.
4. No external fragmentation; may have **internal** fragmentation (last page).
5. User does not see pages; sharing is by pages.

Segmentation

1. Program is split into variable-size logical **segments** (code, stack, modules).
2. Address = (segment number, offset).
3. Segment table stores **base + limit**.
4. No internal fragmentation; may have **external** fragmentation.
5. Matches the programmer's view; whole segments can be shared easily.

Page table: maps logical page number → physical frame number so the process can sit in non-contiguous RAM.

Page fault: needed page is in the address space but **not in RAM**; OS loads it and restarts the instruction.

Demand paging: load a page only when referenced — unused pages stay on disk; more multiprogramming; programs can be larger than RAM.

Thrashing: too much paging, almost no useful work.

6.3 Page Replacement ★★★

For Exam

Include: why replacement is needed; **three algorithms**; Belady's anomaly; then the step-by-step frames for the given string.

When no free frame exists, a victim page is chosen.

1. **FIFO** — replace the oldest page. May show **Belady's anomaly** (more frames can increase faults).
2. **Optimal** — replace the page used farthest in the future. Best fault rate; needs future knowledge.
3. **LRU** — replace the least recently used page.

Method: fill empty frames first (all faults). On a miss, replace by the rule. Hits are not faults.

Practised string, 3 frames: 9, 2, 3, 4, 1, 2, 3, 5, 1, 0, 5, 9, 9, 0, 1

FIFO = **11**, Optimal = **8**, LRU = **12**.

Show each reference, the frames after it, and whether it is a fault or a hit.

6.4 File Allocation Methods ★★

For Exam

Include **all three methods**, each with how it works + advantages + disadvantages.

1. Contiguous — consecutive blocks; directory stores start + length.

Advantage: simple, fast sequential and direct access.

Disadvantage: external fragmentation; size often declared at creation.

2. Linked — linked list of blocks; directory points to first block.

Advantage: no external fragmentation; file can grow.

Disadvantage: pointer overhead; mainly sequential; pointer loss can truncate the file.

3. Indexed — all block addresses in an **index block**; directory points to the index.

Advantage: direct access; no external fragmentation.

Disadvantage: index-block overhead; large files need multilevel index.

)